

UNIDAD 6. ANEXO I

POO: EXCEPCIONES

Programación
CFGS DAW

Adaptado por: José Miguel Blázquez
Autores: Carlos Cacho y Raquel Torres
Revisado por: Lionel Tarazona y Encarni Ferrero

##\$%&##\$##

Licencia



Reconocimiento - NoComercial - CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

'o (enclatura

A lo largo de este te (a se u)lizarán dis)ntos s* (+olos para dis)nguir ele (entos i (portantes dentro del contenido, !stos s* (+olos son:

 - (portante

 Atenci n

 -nteresante

ÍNDICE DE CONTENIDO

1. Introducción.....	4
1.1 Ejemplo 1.....	5
1.2 Ejemplo 2.....	6
1.3 Ejemplo 3.....	7
2. Lanzar excepciones (throw).....	8
2.1 ¿Por qué lanzar excepciones?.....	8
2.2 Cómo lanzar una excepción.....	8
2.3 Indicar la excepción en la cabecera del método.....	9
2.4 Lanzando distintos tipos de excepciones.....	10
2.5 Las excepciones se lanzan de método a método (la patata caliente).....	10
3. Manejar excepciones (try – catch – finally).....	11
3.1 Manejadores de excepciones.....	11
3.2 Ejemplo 4.....	12
3.3 Particularidades de la cláusula <i>catch</i>	12
3.4 Cláusulas <i>catch</i> múltiples.....	13
3.5 Ejemplo 5.....	14
3.6 El objeto <i>Exception</i>	15
4. Jerarquía y tipos de excepciones Java.....	16
5. Definir excepciones propias.....	18
5.1 Ejemplo 6.....	18
6. Agradecimientos.....	20

UD6-ANEXO1. POO: EXCEPCIONES

%, - 'TR. / OCC-1'

Una excepción es un error semántico que se produce en tiempo de ejecución, a saber (os que aunque un código sea correcto sintácticamente es código Java válido y puede compilarse es posible que durante su ejecución se produzcan errores **inesperados** como por ejemplo:

- División por cero,
- Intentar acceder a una posición de un array fuera de sus límites,
- Al llamar al método de un Scanner el usuario no introduce un valor entero,
- Intentar acceder a un fichero que no existe o que está en un disco duro corrupto,
- ...etc,

Cuando esto ocurre la Máquina virtual Java crea un objeto de la clase **Exception** las excepciones en Java son objetos y se notifica el hecho al sistema de ejecución, este dice que se ha lanzado una excepción 4: **Throwing Exception**:5, existen también los errores internos que son objetos de la clase **Error** que no estudiaremos, Ambas clases **Error** y **Exception** son clases derivadas de la clase base **Throwable**,

Un método se dice que es capaz de tratar una excepción 4: **Catch Exception**:5 si ha previsto el error que se ha producido y las operaciones a realizar para recuperar el programa de ese estado de error, o es suficiente capturar la excepción si el error no se trata tan solo conseguir (os que el programa no se pare pero el error puede provocar que los datos o la ejecución no sean correctos,

En el momento en que es lanzada una excepción la Máquina virtual Java recorre la pila de llamadas de métodos en busca de alguno que sea capaz de tratar la clase de excepción lanzada, para ello comienza examinando el método donde se ha producido la excepción si este método no es capaz de tratarla examina el método desde el que se realizó la llamada al método donde se produjo la excepción y así sucesivamente hasta llegar al método o de ellos, en caso de que ninguno de los métodos de la pila sea capaz de tratar la excepción la Máquina virtual Java muestra un mensaje de error y el programa termina,

Los programas escritos en Java también pueden lanzar excepciones explícitamente mediante la instrucción **throw** lo que facilita la devolución de un código de error al método que invocó el método que causó el error,

%,% !é (plo %

Co (o pri (er encuentro con las e8cepciones3 va (os a e8ecutar el siguiente progra (a, !n él va (os a 7orzar una e8cepci n al intentar dividir un n= (ero entre \$:

```

12 public class Ejemplos_excepciones {
13
14     public static void main(String[] args) {
15         int div, x, y;
16
17         x = 3;
18         y = 0;
19
20         div = x / y;
21
22         System.out.println("El resultado es " + div);
23     }
24
25 }

```

"iendo la salida:

```

run:
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at ejemplos_excepciones.Ejemplos_excepciones.main(Ejemplos_excepciones.java:20)
    C:\Users\user\AppData\Local\NetBeans\Cache\8.2\executor-snippets\run.xml:53: Java returned: 1
BUILD FAILED (total time: 0 seconds)

```

Lo que ha ocurrido es que la (áquina virtual Java 4el progra (a que e8ecuta c digo Java5 ha detectado una condici n de error3 la divisi n por \$3 y ha creado un o+eto de la clase *java.lang.ArithmeticException*, Co (o el (étodo donde se ha producido la e8cepci n no es capaz de tratarla3 la (áquina virtual Java 9naliza el progra (a en la l*nea #\$ y (uestra un (ensa8e de error con la in7or (aci n so+re la e8cepci n que se ha producido,

%,# !e (plo #

A continuación vamos a forzar una excepción de conversión, para ello vamos a intentar pasar a entero una cadena que no lleva solo caracteres numéricos:

```

12 public class Ejemplos_excepciones {
13
14     public static void main(String[] args) {
15         String cadena = "56s";
16         int num;
17
18         num = Integer.parseInt(cadena);
19
20         System.out.println("El número es " + num);
21     }
22
23 }

```

viendo la salida:

```

run:
Exception in thread "main" java.lang.NumberFormatException: For input string: "56s"
    at java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)
    at java.lang.Integer.parseInt(Integer.java:580)
    at java.lang.Integer.parseInt(Integer.java:615)
    at ejemplos_excepciones.Ejemplos_excepciones.main(Ejemplos_excepciones.java:18)
C:\Users\user\AppData\Local\NetBeans\Cache\8.2\executor-snippets\run.xml:53: Java returned: 1
BUILD FAILED (total time: 0 seconds)

```

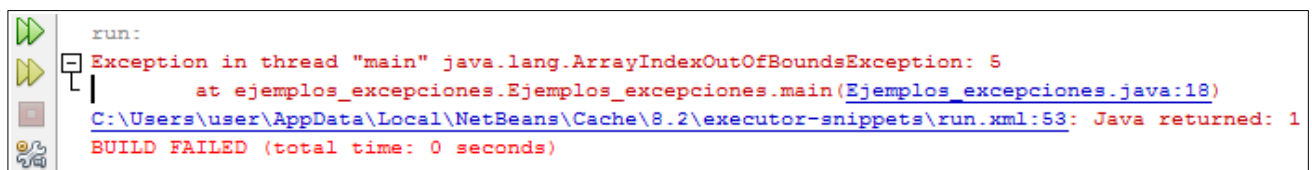
Veamos a que la cadena no tiene el formato adecuado 4>?@sA no representa un número válido el método Integer.parseInt no puede convertirla a un valor de tipo int y lanza la excepción NumberFormatException, La máquina virtual Java analiza el programa en la línea 18 y muestra por pantalla la información sobre la excepción que se ha producido,

%D !e (plo D

!n este eíe (plo va (os a 7orzar una e8cepci n de l* (ites del vector3 para ello va (os a crear un vector e intentar acceder a una posici n que no e8iste:

```
12 public class Ejemplos_excepciones {
13
14     public static void main(String[] args) {
15         int v[] = {1,2,3};
16         int elem;
17
18         elem = v[5];
19
20         System.out.println("El elemento es " + elem);
21
22     }
23
24 }
```

"iendo la salida:



```
run:
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 5
    at ejemplos_excepciones.Ejemplos_excepciones.main(Ejemplos_excepciones.java:18)
    C:\Users\user\AppData\Local\NetBeans\Cache\8.2\executor-snippets\run.xml:53: Java returned: 1
BUILD FAILED (total time: 0 seconds)
```

Al intentar acceder a una posici n que so+repasa el ta (aEo del vector se produce una e8cepci n de)po *ArrayIndexOutOfBoundsException*, La (áquina virtual de áava 9naliza el progra (a en la l*nea %C y (uestra el (ensaíe de error so+re la e8cepci n que se ha producido,

#, LA ' FAR !GC! ;C- . ' !" 4THR . I 5

#, % J; or qué lanzar e8cepcionesK

On progra (ador puede construir su c digo de 7or (a que se lancen e8cepciones cuando se intente hacer algo incorrecto o inesperado 4en ocasiones es reco (enda+le5, ;or e8e (plo3 cuando los argu (entos que se le pasan a un (étodo no son correctos o no cu (plen ciertos criterios,

!sto es ha+itual en la ;rogra (aci n . rientada a . +8etos 4; . . 5, Recuerda que una clase de+e ser la responsa+le de la l gica de sus o+8etos: asegurar que los datos sean válidos3 ade (ás de controlar qué está per (i)do y qué no está per (i)do hacer, **Por ejemplo, si se instancia una clase con valores incorrectos:** co (o un o+8eto ;ersona con un / ' - no válido3 una edad nega)va3 una cuenta +ancaria con saldo inicial nega)vo3 etc, !n esos casos **es conveniente que el constructor lance una excepción,**

Ta (+ién **puede ser apropiado lanzar excepciones en los setters** si el valor no es válido 4i (portancia del encapsula (iento y la ocultaci n53 **y en cualquier otro método en el que se intente hacer algo no permitido o que viole la integridad del objeto** co (o por e8e (plo re)rar dinero de una cuenta sin saldo su9ciente,

✎ Téngase en cuenta que **las excepciones pueden manejarse y controlarse sin que el programa se pare** 4en el apartado D se e8plica c (o5,

!s decir3 lanzar una e8cepci n no i (plica necesaria (ente que el progra (a ter (inará3 es si (ple (ente una 7or (a de avisar de un error, **Quien llame al método es responsable de manejar la excepción para que el programa no se pare,**

C (o lanzar una e8cepci n

;ara lanzar (anual (ente una e8cepci n se u)liza la pala+ra reservada **throw** seguido de un o+8eto de)po *Exception* 4o alguna de sus su+clases co (o *ArithmeticException*, *NumberFormatException*, *ArrayIndexOutOfBoundsException*, etc.5, Co (o las e8cepciones son o+8etos3 de+en instanciarse con el operador >neLA, ;or lo tanto3 **podemos lanzar una excepción genérica así:**

```
throw new Exception();
```

!sto es equivalente a: pri (ero instanciar el o+8eto !8cep)on3 y luego lanzarlo:

```
Exception e = new Exception();
throw e;
```

!l constructor de !8cep)on per (ite 4opcional (ente5 un argu (ento "tring para dar detalles so+re el pro+le (a, "i la e8cepci n no se (aneña y el progra (a se para3 el (ensa8e de error se (ostrará por la consola 4esto es (uy =)l para depurar progra (as5,


```
throw new Exception("La edad no puede ser negativa");
```

En lugar de lanzar excepciones genéricas como la (+ién es posible lanzar excepciones específicas de Java como por ejemplo `ArrayIndexOutOfBoundsException`, `ArithmeticException`, `NumberFormatException`, etc. En Java todas las clases de excepciones heredan de `Exception`,

```
throw new NumberFormatException("...");
```

✎ En lugar de lanzar excepciones propias de Java como `ArithmeticException` o `NumberFormatException` etc., **normalmente es preferible lanzar excepciones genéricas 'Exception' o mejor aún, utilizar nuestras propias excepciones** (apartado @5,

#D -ndicar la excepción en la cabecera del método

Es obligatorio indicar en la cabecera del método que puede lanzar excepciones, para ello hay que añadir a la derecha de la cabecera y antes de las llaves la palabra reservada **throws** seguido del **tipo de excepción** que puede lanzar (si puede lanzar distintos tipos de excepciones deben indicarse todas separadas por comas),

por ejemplo veamos el método `setEdad(int edad)` de la clase `Persona`, como hemos decidido que la edad de una persona no puede ser negativa lanzaremos una excepción si `edad < 0`,

```
// Setter del atributo edad. Debe ser >= 0
public void setEdad(int edad) throws Exception {
    if (edad < 0)
        throw new Exception("Edad negativa no permitida");
    else
        this.edad = edad;
}
```

Hay que tener en cuenta que **al lanzar una excepción se parará la ejecución de dicho método (no se ejecutará el resto del código del método) y se lanzará la excepción al método que lo llamó**, por ejemplo desde la función `main` (al llamar a `setEdad`) como puede suceder que `setEdad` lance una excepción entonces en la práctica es posible que el `main` lance una excepción (no directa (ente con un `throw`) sino por la excepción que nos lanza `setEdad`), por lo tanto también tiene (os que especificar en el `main` que se puede lanzar una excepción:

```
public static void main(String[] args) throws Exception {
    Persona p = new Persona("44193900L", "Pepito", 27);
    ...
    ...
}
```

```
p.setEdad(valor); // Puede lanzar una excepción
}
```

#,0 Lanzando distintos tipos de excepciones

Un método puede lanzar distintos tipos de excepciones si lo considera (o no necesario), en tal caso hay que especificar todos los tipos posibles en la cabecera, separados por comas, ; o else (plaza) (agrega) (os que el constructor de Persona toma como argumento el dni y la edad y quiere (os lanzar excepciones distintas segun cada caso,

```
public Persona(String dni, int edad) throws InvalidDniException, InvalidEdadException {
    if (!dni.matches("[0-9]{8}[A-Z]")) {
        throw new InvalidDniException("DNI no válido: " + dni);
    }
    if (edad < 0) {
        throw new InvalidEdadException("Edad no válida: " + edad);
    }
    this.dni = dni;
    this.edad = edad;
}
```

 . Así téngase en cuenta que *InvalidEdadException* e *InvalidDniException* no son tipos de excepciones de Java sino excepciones propias que nosotros hemos creado (o adaptado) para nuestro software (ver apartado 5,

#,? Las excepciones se lanzan de (modo a (modo la patata caliente

. Así se ve que el lanzamiento de excepciones se produce recursivamente a través de la secuencia de llamadas a (métodos, (agrega) (os que en el (método A llama (os al (método B que llama (os al (método C etc, hasta llegar a !,

A → B → C → / → ! (secuencia de llamadas de (métodos)

"i el (método ! lanza una excepción que esta le llegará a / que a su vez la lanzará a C etc, recorriendo el camino inverso hasta llegar al (método inicial A,

A ← B ← C ← / ← R ! (secuencia de lanzamiento de excepciones)

;or lo tanto (o todos estos (métodos pueden acabar lanzando una excepción en sus cabezas habrá que incluir el **throws Exception** o el que corresponda segun el tipo de excepción,

 ¡Ojo! No es recomendable dejar que las Excepciones de nuestro software lleguen de forma descontrolada hasta el main y terminar (en el programa),
Lo ideal es manejar las excepciones (o ver (os en el siguiente apartado,

MANEJADORES DE EXCEPCIONES

Manejadores de excepciones

En Java se pueden manejar excepciones utilizando tres mecanismos: los llamados **manejadores de excepciones**, existen tres y funcionan conjuntamente:

- Bloque **try** 4intentar5: código que podrá lanzar una excepción,
- Bloque **catch** 4capturar5: código que manejará la excepción si es lanzada,
- Bloque **finally** 4final (ente5: código que se ejecuta tanto si hay excepción como si no,



Un **manejador de excepciones** es una bloque de código encargado de tratar las excepciones para intentar recuperarse del fallo y **evitar que la excepción sea lanzada descontroladamente hasta el main y termine el programa**,

Antes de que se utilice un **try** es obligatorio utilizar al menos un **catch**, el **finally** es opcional,

```
try {
    // Instrucciones que podrían lanzar una excepción.
}
catch (TipoExcepción nombreVariable) {
    // Instrucciones que se ejecutan cuando 'try' lanza una excepción.
}
finally {
    // Instrucciones que se ejecutan tanto si hay excepción como si no.
}
```

El bloque try intentará ejecutar el código, si se produce una excepción se abandona dicho bloque y no se ejecutarán las demás instrucciones del try y se saltará al bloque catch, Lógicamente si en el try no se produce ninguna excepción el bloque catch se ignora,

El bloque catch capturará las excepciones del tipo *TipoExcepción*, evitando que sea lanzada al método que nos llama. Aquí definiremos/escribiremos las instrucciones que sean necesarias para manejar el error, pueden especificarse varios bloques **catch** para distintos tipos de excepciones,

El bloque finally es opcional y se ejecutará tanto si se ha lanzado una excepción como si no,

Tea es un ejemplo sencillo,

D, # !e (plo 0

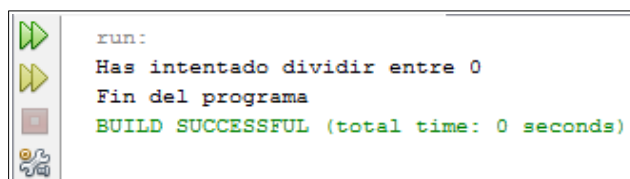
Ta (os a producir una e8cepci n y tratarla haciendo uso de los (aneñadores *try-catch*:

```

12 public class Ejemplos_excepciones {
13
14     public static void main(String[] args) {
15         int x = 1, y = 0;
16
17         try
18         {
19             int div = x / y;
20
21             System.out.println("La ejecución no llegará aquí.");
22         }
23         catch(ArithmeticException ex)
24         {
25             System.out.println("Has intentado dividir entre 0");
26         }
27
28         System.out.println("Fin del programa");
29     }
30 }

```

"iendo la salida:



```

run:
Has intentado dividir entre 0
Fin del programa
BUILD SUCCESSFUL (total time: 0 seconds)

```

Al intenta dividir por cero se lanza auto (á) ca (ente una *ArithmeticException*, Co (o esto sucede dentro del +loque try la eñecuci n del progra (a pasa al pri (er +loque *catch* porque coincide con el)po de e8cepci n producida 4*ArithmeticException*5, "e eñecutará el c digo del +loque catch y luego el progra (a con) nuará con nor (alidad,

D,D ;ar)cularidades de la cláusula *catch*

!s i (portante entender que **el bloque catch solo capturará excepciones del tipo indicado**. "i se produce una e8cepci n dis)nta no la capturará, "in e (+argo **capturará excepciones heredadas del tipo indicado**, ;or eñe (plo3 *catch (ArithmeticException e)* capturará cualquier)po de e8cepci n que herede de Arith (e) c!8cep)on, !l caso (ás general es *catch (Exception e)* que capturará todo)po de e8cepciones porque **en Java todas las excepciones heredan de Exception**,

"in e (+argo3 es (eñor u)lizar e8cepciones lo (ás cercanas al)po de error previsto3 ya que lo que se pretende es recuperar el progra (a de alguna condici n de error y si :se (eten todas las e8cepciones en el (is (o saco:3 segura (ente ha+rá que averiguar después qué condici n de error se produñó para poder dar una respuesta adecuada,

El objetivo de una cláusula *catch* es resolver la condición excepcional para que el programa pueda continuar como si el error nunca hubiera ocurrido.

D.0 Cláusulas *catch* (=l)ples

Se pueden especificar varias cláusulas *catch* tantas como quieras para que cada una capture un tipo diferente de excepción,

```
try {
    // instrucciones que pueden producir distintos tipos de Excepciones
}
catch (TipoExcepción1 e1) {
    // instrucciones para manejar un TipoExcepción1
}
catch (TipoExcepción2 e2) {
    // instrucciones para manejar un TipoExcepción2
}
...
}
catch (TipoExcepciónN eN) {
    // instrucciones para manejar un TipoExcepciónN
}
finally { // opcional
    // instrucciones que se ejecutarán tanto si hay excepción como si no
}
```

Cuando se lanza una excepción dentro del *try* se comprueba cada sentencia *catch* en orden y se ejecuta la primera cuyo tipo coincida con la excepción lanzada. Los demás bloques *catch* serán ignorados. Luego se ejecutará el bloque *finally* si se ha definido y el programa continuará su ejecución después del bloque *try-catch-finally*.

⚡ "Si la excepción producida no coincide con ninguno de los *catch* entonces la excepción será lanzada al método que nos llama (,

Tea (os un ejemplo con cláusulas *catch* (=l)ples,

D,? !e (plo ?

Ta (os a eñecutar dis)ntas instrucciones dentro del +loque *try* y (aneñare (os las eñcepciones de divisi n por cero y de si so+repasa (os el ta (aEo del vector,

```

14 public class Ejemplos_excepciones {
15
16     public static void main(String[] args) {
17         int x, y, div, pos;
18         int[] v = {1,2,3};
19         Scanner in = new Scanner(System.in);
20
21         try
22         {
23             System.out.print("Introduce el numerador: ");
24             x = in.nextInt();
25
26             System.out.print("Introduce el denominador: ");
27             y = in.nextInt();
28
29             div = x / y;
30
31             System.out.println("La división es " + div );
32
33             System.out.print("Introduce la posición del vector a consultar: ");
34             pos = in.nextInt();
35
36             System.out.println("El elemento es " + v[pos] );
37
38         }
39         catch(ArithmeticException ex)
40         {
41             System.out.println("División por cero: " + ex);
42         }
43         catch(ArrayIndexOutOfBoundsException ex)
44         {
45             System.out.println("Sobrepasado el tamaño del vector: " + ex);
46         }
47
48         System.out.println("Fin del programa");
49     }
50 }

```

;ueden suceder tres cosas diñerentes:

- !! *try* se eñecuta sin eñcepciones} se ignoran los *catch* y se i (pri (e >Min del progra (aA,
- "e produce la eñcepci n de divisi n por cero 4!*nea #U53 el Vuño de eñecuci n salta al %er *catch*} se i (pri (e el (ensañe >/ivisi n por cero,,A y luego >Min del progra (aA,
- "e produce la eñcepci n de so+repasar el vector 4!*nea D@53 el Vuño de eñecuci n salta al #W *catch*} se i (pri (e el (ensañe >"o+repasado el ta (aEo del vector,,A y >Min del progra (aA,

D, @ !l o+ñeto !8cep)on

Toda excepción genera un objeto de la clase `Exception` o uno (más específico que hereda de `Exception`), /icho objeto contendrá detalles sobre el error producido, ;uede ser interesante (strar esta información para que la vea el usuario 4que sepa qué ha sucedido5 o el desarrollador 4para depurar y corregir el código si es necesario5, !n la cláusula `catch` tiene (os acceso al objeto en caso de que quiera (os u)lizarlo:

Los dos (étodos de `Exception` (ás =)les son:

- **`getMessage()`** Q /evuelve un `String` con un texto si (ple sobre el error,
- **`printStackTrace()`** Q !s el que (ás información (aci n proporciona, -ndica qué)po de excepción se ha producido3 el (ensañe si (ple3 y ta (+ién toda la pila de llamadas, !sto es lo que hace Java por defecto cuando una excepción no se (aneña y acaba parando el programa,

```
...
catch (Exception e){

    // Mostramos el mensaje de la excepción
    System.err.println("Error: " + e.getMessage());

    // Mostramos toda la información, mensaje y pila de llamadas
    e.printStackTrace();
}
...
```

Los objetos de)po `Exception`)enen so+recargado el (étodo `toString()` por lo que ta (+ién es posible i (pri (irlos directamente (ediante `println()`,

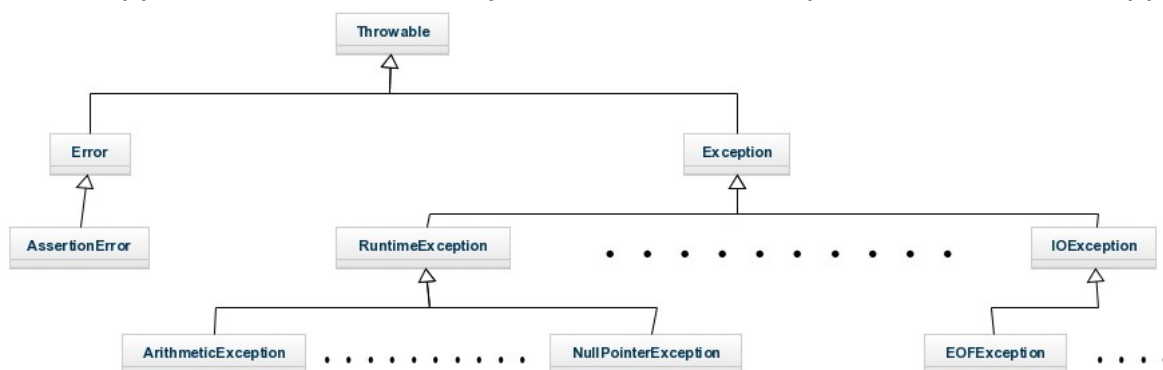
```
...
catch (Exception e){

    // Mostramos el mensaje de la excepción
    System.out.println(e);

}
...
```

0, J! RARXOYA 2 T-; . " / ! ! GC! ; C- . ' ! " JATA

La clase `Exception` hereda de `Throwable` y a su vez todas las excepciones heredan de `Exception`,



Código `java.lang` es incluido (portado de `Throwable`) y citado en todos los programas (la mayor parte de las excepciones derivadas de `RuntimeException` están disponibles de `Throwable` automáticamente, **Ade más no es necesario incluirlas en ninguna cabecera de método mediante `throws`**,

Las excepciones pueden ser comprobadas y no comprobadas:

- **Excepciones comprobadas:** aquellas que Java comprueba durante la compilación antes de la ejecución del programa,
- **Excepciones no comprobadas:** aquellas que Java no puede comprobar durante la compilación y se producirán durante la ejecución del programa,

Las **excepciones comprobadas** definidas en `java.lang` son:

Excepción	Significado
<code>ClassNotFoundException</code>	No se ha encontrado la clase,
<code>CloneNotSupportedException</code>	Intento de duplicado de un objeto que no implementa la interfaz <code>Cloneable</code> ,
<code>IllegalAccessException</code>	Se ha denegado el acceso a una clase,
<code>InstantiationException</code>	Intento de crear un objeto de una clase abstracta o interfaz,
<code>InterruptedException</code>	Hilo interrumpido por otro hilo,
<code>NoSuchFieldException</code>	El campo solicitado no existe,
<code>NoSuchMethodException</code>	El método solicitado no existe,

Las **subclases de *RuntimeException* no comprobadas** son:

Excepción	Significado
<i>AithmeticException</i>	! rror arit (é)co co (o divisi n entre cero,
<i>ArrayIndexOutOfBoundsException</i>	Índice de la (atriz 7uera de su l* (ite,
<i>ArrayStoreException</i>	Asignaci n a una (atriz de)po inco (pa)+le,
<i>ClassCastException</i>	Conversi n invalida,
<i>IllegalArgumentException</i>	Oso inválido de un argu (ento al lla (ar a un (étodo,
<i>IllegalMonitorStateException</i>	. peraci n de (onitor inválida3 co (o esperar un hilo no +loqueado,
<i>IllegalStateException</i>	! l entorno o aplicaci n están en un estado incorrecto,
<i>IllegalThreadStateException</i>	La operaci n solicitada es inco (pa)+le con ell estado actual del hilo,
<i>IndexOutOfBoundsException</i>	Alg=n)po de *ndice está 7uera de su rango o de su l* (ite,
<i>NegativeArraySizeException</i>	La (atriz)ene un ta (aEo nega)vo,
<i>NullPointerException</i>	Oso incorrecto de una re7erencia ' OLL,
<i>NumberFormatException</i>	Conversi n incorrecta de una cadena a un 7or (ato nu (érico,
<i>SecurityException</i>	-ntento de violaci n de seguridad,
<i>StringIndexOutOfBounds</i>	-ntento de so+repasar el l* (ite de una cadena,
<i>TypeNotPresentException</i>	Tipo no encontrado,
<i>UnsupportedOperationException</i>	. peraci n no ad (i)da,

 ¡s interesante conocer los dis)ntos)pos de e8cepciones3 pero **no es necesario sabérselas de memoria ni entender exactamente cuándo se produce cada una,**

Aunque las e8cepciones que incorpora Java ges)onan la (ayor*a de los errores (ás co (unes3 es pro+a+le que el progra(ador pre9era crear sus propios)pos de e8cepciones para ges)onar situaciones espec*9cas de sus aplicaciones, Tan solo hay que de9nir una su+clase de *Exception*3 que natural (ente es su+clase de *Throwable*,

?, / !M-'-R !GC! ;C- . ' !" ;R. ; -A"

Cuando desarrollas (o simplemente al desarrollar nuestras propias clases) es habitual que se puedan producir excepciones que no estén definidas dentro del lenguaje Java, para crear una excepción propia tienes que definir una clase derivada de la clase base *Exception*,

El uso de esta nueva excepción es el mismo que el que hemos visto,

?,% !e (plo @

También vamos a ver un ejemplo sencillo de definición y uso de un nuevo tipo de excepción llamada *ExcepcionPropia* que utilizaremos cuando a se le pase a *metodo* un valor superior a 10,

El código y el método que lanza la excepción:

```

14 public class Ejemplos_excepciones {
15
16     public static void main(String[] args) {
17         try
18         {
19             metodo(1);
20             metodo(20);
21         }
22         catch (ExcepcionPropia e)
23         {
24             System.out.println("capturada : " + e);
25         }
26     }
27
28     static void metodo(int n) throws ExcepcionPropia
29     {
30         System.out.println("Llamado por metodo(" + n + ")");
31
32         if (n > 10)
33             throw new ExcepcionPropia(n);
34
35         System.out.println("Finalización normal");
36     }
37 }

```

La definición de la nueva excepción,

```

12  public class ExcepcionPropia extends Exception
13  {
14      private int num;
15
16      ExcepcionPropia(int n)
17      {
18          this.num = n;
19      }
20      public String toString()
21      {
22          return "Excepcion Propia[" + this.num + "]";
23      }
24  }
25

```

viendo la salida:

```

run:
Llamado por metodo(1)
Finalización normal
Llamado por metodo(20)
capturada :Excepcion Propia[20]
BUILD SUCCESSFUL (total time: 0 seconds)

```

Cuando se puede observar la excepción se lanza cuando el número es mayor que 10 y será tratada por la nueva clase creada que hereda de Excepcion, Además se sobrescribe el método toString que es el encargado de mostrar el mensaje asociado a la excepción,

@, A\RA\!C-M-! 'T. "

Apuntes actualizados y adaptados al C++ / C# a partir de la siguiente documentación:

Apuntes ; rogra (ación de José Antonio / *az_Alejo, -!" Ca (p de Morvedre,